

The lux Display Patch-Application Boundary: a Z Specification

Formal model of the crash-free patch discipline

July 2026

1 Introduction

An agent sends the display a batch of update patches. Each patch names an element by id and either removes it or sets some of its fields. A patch may be wrong in several ways: the value it sets may be out of range or not a number, the field it names may not exist on the element, or the id it names may not exist in the scene. The display processes the batch in a single message-loop turn. The one property that must hold is that *no patch, however malformed, terminates that loop*.

The same class of defect — a patch that terminates the display — recurred across four fix/review rounds:

1. a crash on an out-of-range **ValueError** from a setter;
2. a crash on a **TypeError** from a setter given the wrong type;
3. a non-atomic partial mutation, where a multi-field patch applied some fields before a later field was rejected;
4. a crash on an unknown-field structural error, where the field-name check raises *before* the value-rejection **try** and the exception propagates through **apply_update** to the message loop and terminates the display.

Rounds 1 and 2 were closed by catching (**ValueError**, **TypeError**) around the setter call; round 3 by an all-or-nothing snapshot/rollback in **Element.apply_patch**; round 4 by surfacing an unknown field per patch — logging and skipping it — rather than letting it raise out of **_apply_patch_set** and kill the loop. All four are now fixed; each survives only as a fidelity variant (Section 10) that reintroduces the defect to confirm the model still detects it. Empirical case-by-case patching kept missing cases, so this document models the boundary as a finite state machine over a bounded batch, letting ProB enumerate every patch class exhaustively.

2 Basic Types

[*ELEM*]

ELEM is the set of element ids that may appear in the scene or be named by a patch. Two ids suffice to exhibit batch-continuation across distinct targets (one patch rejected, a later patch to another element applied); the carrier is bounded by ProB's **DEFAULT_SETSIZE**.

3 Free Types

VALID ::= *good* | *bad*

Whether an element's tracked field value satisfies that field's invariant. **good**: in range, correct type. **bad**: out of range, NaN, or wrong type — the value a setter rejects. The safety goal is that a **bad** value is never *installed*.

LIVE ::= *alive* | *terminated*

The display message loop. **alive**: still processing messages. **terminated**: an unhandled exception escaped the patch path and killed the loop — the defect this model exists to forbid.

$$KIND ::= setGood \mid setBadVal \mid setBadType \mid setUnknown \mid removeIt$$

The five ways a patch acts on its target. **setGood**: set a known field to a valid value. **setBadVal**: set a known field to an out-of-range or NaN value (a setter **ValueError**). **setBadType**: set a known field to a value of the wrong type (a setter **TypeError**). **setUnknown**: name a field the element does not have (the structural **ValueError** raised before the value-rejection **try**). **removeIt**: remove the element.

A patch is a token carrying its kind and its target id:

$$\begin{aligned} TOKEN ::= & tSetGood\langle ELEM \rangle \mid tSetBadVal\langle ELEM \rangle \\ & \mid tSetBadType\langle ELEM \rangle \mid tSetUnknown\langle ELEM \rangle \\ & \mid tRemove\langle ELEM \rangle \end{aligned}$$

Each constructor is an injection $ELEM \hookrightarrow TOKEN$, so a token determines its target and its kind uniquely. Two total projections recover them:

$\begin{aligned} & targetOf : TOKEN \rightarrow ELEM \\ & kindOf : TOKEN \rightarrow KIND \end{aligned}$
$\begin{aligned} & \forall e : ELEM \bullet \\ & \quad targetOf(tSetGood\ e) = e \wedge targetOf(tSetBadVal\ e) = e \wedge \\ & \quad targetOf(tSetBadType\ e) = e \wedge targetOf(tSetUnknown\ e) = e \wedge \\ & \quad targetOf(tRemove\ e) = e \\ & \forall e : ELEM \bullet \\ & \quad kindOf(tSetGood\ e) = setGood \wedge kindOf(tSetBadVal\ e) = setBadVal \wedge \\ & \quad kindOf(tSetBadType\ e) = setBadType \wedge \\ & \quad kindOf(tSetUnknown\ e) = setUnknown \wedge kindOf(tRemove\ e) = removeIt \end{aligned}$

Because every **TOKEN** is exactly one construction, these equations pin **targetOf** and **kindOf** to a unique total function apiece.

The batch is a bounded sequence of tokens. The bound keeps the carrier finite so ProB can enumerate every ordering:

$\begin{aligned} & maxlen : \mathbb{N} \\ & maxlen = 3 \end{aligned}$

4 State

The state is flat: one schema, five components. Its predicate carries only *structural* invariants — that the tracked-value map is defined exactly on the present elements, that a partially-mutated element is present, and the length bound on the queue. These are coherence constraints, true in every design. The four safety properties (Section 8) are deliberately *absent* here, so a violating state stays reachable and ProB can find it.

$\begin{aligned} & PatchSys \\ & live : LIVE \\ & present : \mathbb{P}\ ELEM \\ & prim : ELEM \leftrightarrow VALID \\ & partial : \mathbb{P}\ ELEM \\ & queue : seq\ TOKEN \end{aligned}$
$\begin{aligned} & dom\ prim = present \\ & partial \subseteq present \\ & \#queue \leq maxlen \end{aligned}$

present is the set of element ids currently in the scene. **prim** records, for each present element, whether its tracked field holds a **good** or **bad** value. **partial** is a history component: the set of elements a *rejected* patch left partly mutated. In a correct design it is always empty; it is the device that turns the two-state atomicity property into a one-state invariant ProB can check. **queue** is the unprocessed tail of the current batch.

5 Initialization

<i>Init</i>	
<i>PatchSys'</i>	
$live' = alive$	
$present' = ELEM$	
$prim' = (\lambda e : ELEM \bullet good)$	
$partial' = \emptyset$	
$queue' = \langle \rangle$	

A single initial state: the loop is alive, every element is present with a **good** value, nothing is partially mutated, and the batch is empty.

6 Building a Batch

An agent enqueues patches until the batch is full. Any token may be enqueued — including one whose target is not present, which models a patch for a missing id:

<i>Enqueue</i>	
$\Delta PatchSys$	
$p? : TOKEN$	
$\#queue < maxlen$	
$queue' = queue \frown \langle p? \rangle$	
$live' = live$	
$present' = present$	
$prim' = prim$	
$partial' = partial$	

7 Applying a Patch

Each step processes the head of the queue. The five operation schemas partition every head token by target-presence and kind, so whenever the loop is alive and the queue is non-empty exactly one is enabled: the processing of a batch never gets stuck (Section 8, Invariant BC).

A patch whose target is not present is a no-op — the tree walk returns no location, and the applier warns and skips. This covers both a set and a remove for a missing id:

<i>ApplyMissing</i>	
$\Delta PatchSys$	
$live = alive$	
$queue \neq \langle \rangle$	
$targetOf(head\ queue) \notin present$	
$queue' = tail\ queue$	
$live' = live$	
$present' = present$	
$prim' = prim$	
$partial' = partial$	

A remove of a present element detaches it: the id leaves **present**, its value map entry and any partial mark go with it.

ApplyRemove

$\Delta PatchSys$

$live = alive$
 $queue \neq \langle \rangle$
 $targetOf(head\ queue) \in present$
 $kindOf(head\ queue) = removeIt$
 $present' = present \setminus \{targetOf(head\ queue)\}$
 $prim' = \{targetOf(head\ queue)\} \triangleleft prim$
 $partial' = partial \setminus \{targetOf(head\ queue)\}$
 $queue' = tail\ queue$
 $live' = live$

A set of a valid value on a known field installs **good**:

ApplySetGood

$\Delta PatchSys$

$live = alive$
 $queue \neq \langle \rangle$
 $targetOf(head\ queue) \in present$
 $kindOf(head\ queue) = setGood$
 $prim' = prim \oplus \{targetOf(head\ queue) \mapsto good\}$
 $present' = present$
 $partial' = partial$
 $queue' = tail\ queue$
 $live' = live$

A set that a validated setter *rejects* — an out-of-range or NaN value (**ValueError**) or a wrong-typed value (**TypeError**) — is caught around the setter call, logged, and skipped. Nothing is installed, so the **bad** value never reaches **prim**; nothing is mutated, so no element is marked **partial**; the loop stays alive and the batch continues:

ApplyReject

$\Delta PatchSys$

$live = alive$
 $queue \neq \langle \rangle$
 $targetOf(head\ queue) \in present$
 $kindOf(head\ queue) \in \{setBadVal, setBadType\}$
 $queue' = tail\ queue$
 $live' = live$
 $present' = present$
 $prim' = prim$
 $partial' = partial$

A set naming an unknown field is a *structural* error. The correct boundary surfaces it — logs it and/or returns it to the agent — per patch, then continues. It installs nothing, mutates nothing, and, above all, does not terminate the loop:

ApplySurface

$\Delta PatchSys$

live = *alive*

queue $\neq \langle \rangle$

targetOf (*head queue*) $\in present$

kindOf (*head queue*) = *setUnknown*

queue' = *tail queue*

live' = *live*

present' = *present*

prim' = *prim*

partial' = *partial*

In this correct design **ApplyReject** and **ApplySurface** have the same effect — skip and continue. They are kept as separate schemas because they diverge under the buggy variants of Section 10: round 1/2 breaks **ApplyReject**, round 4 breaks **ApplySurface**, and the model must be able to break each independently.

8 Invariants

Four properties must hold across all reachable states. They are stated here as predicates over **PatchSys**; none is placed in the **PatchSys** schema, because a predicate placed there would be enforced as a guard on every successor, silently disabling any operation that would break it and thereby masking the very defect this model exists to catch.

Invariant CF — crash-freedom. The display loop is never terminated:

live = *alive*.

This is the load-bearing property. Every patch class — a rejected value, a rejected type, an unknown field, a missing id, a malformed remove — is handled at the boundary; none escapes to the loop.

Invariant AT — atomicity. No element is ever left partially mutated by a rejected patch:

partial = $\emptyset$.

A rejected multi-field set restores every field it touched, so the “keeps its previous value” claim is literally true.

Invariant VP — validity-preservation. No bad value is ever installed; every present element’s tracked value is good:

$\text{ran } prim \subseteq \{good\}$.

Invariant BC — batch-continuation. A rejected patch never abandons the rest of its batch. Because the only thing that halts processing is termination, and termination never occurs (Invariant CF), the observable failure is a batch left unfinished with the loop dead:

$\neg (live = terminated \wedge queue \neq \langle \rangle)$.

Each of **ApplyReject** and **ApplySurface** reduces the queue by exactly its head and frames **live** at **alive**, so the successor patch is processed on the next step.

9 Verification

Type-checking is a fuzz pass:

```
fuzz -t docs/patch_application.tex
```

Each safety property is checked by asking ProB whether its negation is *reachable*. A goal that is not found means the property holds on every reachable state:

```
# Invariant CF (must report: goal NOT found)
probccli docs/patch_application.tex -model_check -nodead \
  -goal "live=terminated" -p DEFAULT_SETSIZE 2

# Invariant AT (must report: goal NOT found)
probccli docs/patch_application.tex -model_check -nodead \
  -goal "partial /= {}" -p DEFAULT_SETSIZE 2

# Invariant VP (must report: goal NOT found)
probccli docs/patch_application.tex -model_check -nodead \
  -goal "bad : ran(prim)" -p DEFAULT_SETSIZE 2

# Invariant BC (must report: goal NOT found)
probccli docs/patch_application.tex -model_check -nodead \
  -goal "live=terminated & queue /= <>" -p DEFAULT_SETSIZE 2
```

Deadlock-freedom is the state-space search with no goal; the loop is kept live by `Enqueue` while the batch has room and by an `ApplyX` while it is non-empty, so a correct design never deadlocks:

```
# Deadlock-freedom (must report: no deadlock, no invariant violation)
probccli docs/patch_application.tex -model_check -p DEFAULT_SETSIZE 2
```

10 Fidelity: reproducing the four defects

A model that cannot reproduce the known defects proves nothing. Each of the four rounds corresponds to breaking one guard of the correct design; the correct spec above is what remains once all four are fixed.

Round 1 — crash on out-of-range `ValueError`. Remove the catch from the value-rejection path: in `ApplyReject`, for `setBadVal`, replace the frame `live' = live` with `live' = terminated`. The reachability search for Invariant CF then returns *goal found* — a batch containing one `tSetBadVal` token drives `live` to `terminated`.

Round 2 — crash on `TypeError`. The same edit for `setBadType`. A `tSetBadType` token reaches `terminated: goal found`. Rounds 1 and 2 were two review rounds precisely because value-rejection and type-rejection were handled by different `except` clauses; the model shows they are one case class.

Round 3 — non-atomic partial mutation. Model the missing rollback: in `ApplyReject`, replace `partial' = partial` with `partial' = partial ∪ {targetOf(head queue)}` (a multi-field set applied its first field before the second was rejected, and nothing restored it). The loop stays alive, but Invariant AT's goal `partial /= {}` is now *found*.

Round 4 — crash on unknown-field structural error. This defect — since fixed by surfacing the unknown field per patch, logging and skipping it — is reproduced by making `ApplySurface` replace `live' = live` with `live' = terminated`, modelling a field-name check that raises before the value-rejection `try` and propagates through `apply_update` to the message loop. Invariant CF's goal `live=terminated` is then *found*, and so is Invariant BC's goal `live=terminated & queue /= <>` — an unknown-field patch not only kills the loop but strands every patch after it in the batch.

Under the correct design none of these traces exists: `ApplyReject` and `ApplySurface` frame `live` at `alive` and `partial` unchanged, `prim` is only ever set to `good`, and no operation assigns `terminated`. The reachability searches for CF, AT, VP, and BC therefore return *goal not found* for the correct design and *goal found* — the traces above — for the corresponding buggy variant. That difference is the evidence that the model detects the defects the code was changed, round by round, to prevent.